

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 3 – Comparative Analysis
Weightage:	10% of CIA	Total Marks:	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	P. Sai Charan Tej	Reg. No.:	192465048
Section / Batch:		Date:	

Instructions to Students

- Answer the following question. Total Marks: 100.
- Analyze the given questions thoroughly before answering.
- Compare multiple aspects such as methods, results, advantages, and limitations clearly.
- Critically evaluate the strengths, weaknesses, and implications with proper justification.
- Present meaningful insights, interpretations, and well-supported conclusions from the comparisons.

Question Type	Focus Area	Questions
Comparative Analysis	Comparative analysis of Dynamic Programming and Greedy algorithms for optimization problems.	Q1

SECTION A – Comparative Analysis

Q1 Given coins and items, solve Coin Change and Knapsack problems using DP. Compare recurrence relations and table structures. Analyze differences in constraints. Evaluate complexity. Discuss scalability. Generate insights on DP modeling.

Code of Conduct

I P. Sai Charan Tej(192465048)__(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Comparative Analysis					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Selection of Studies/Parameters	20%	Selects highly relevant and diverse studies with well-defined comparison parameters	Selects relevant studies with minor gaps in parameter definition	Limited or partially relevant selection	Poor or inappropriate selection of studies
Comparison & Differentiation	25%	Clearly compares multiple aspects (methods, results, limitations) with strong differentiation	Good comparison with minor gaps	Basic comparison with limited depth	No clear comparison; mostly descriptive
Critical Evaluation	25%	Critically evaluates strengths, weaknesses, and implications with strong justification	Provides evaluation with some justification	Limited evaluation; lacks depth	No critical evaluation provided
Synthesis & Insight Generation	20%	Generates meaningful insights and conclusions from comparisons	Some insights derived from comparison	Limited or unclear insights	No insights generated
Clarity	10%	Information is clearly presented (tables/figures) with logical structure	Generally clear with minor issues	Presentation lacks clarity or structure	Poor presentation and difficult to understand

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Selection of Studies/Parameters	20%				
Comparison & Differentiation	25%				
Critical Evaluation	25%				
Synthesis & Insight Generation	20%				
Clarity	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Algorithm: Coin Change vs Knapsack Problem Using DP

Part A: Coin Change

1. Start.
2. Read the number of coins n .
3. Read the coin values into `coins[]`.
4. Read the target amount.
5. Create a DP array `dp[amount + 1]`.
6. Initialize all values of `dp` to infinity.
7. Set `dp[0] = 0`.
8. For each amount i from 1 to amount:
 - For each coin:
 - If $\text{coin} \leq i$, update:

`dp[i] = min(dp[i], dp[i - coin] + 1)`

9. If `dp[amount]` is infinity, the amount cannot be formed.
10. Otherwise, `dp[amount]` is the minimum number of coins.

Part B: 0/1 Knapsack

1. Read the number of items m .
2. Read the weights of all items.
3. Read the values of all items.
4. Read the knapsack capacity.
5. Create a DP array `knapsack[capacity + 1]` and initialize it to 0.
6. For each item:
 - Traverse the capacity from maximum to the item's weight.
 - Update:
 - `knapsack[w] = max(knapsack[w], knapsack[w - weight] + value)`
7. The value at `knapsack[capacity]` is the maximum achievable value.
8. Display the results.
9. Stop.

Output:



SIMATS Online Programming Lab

DAA PROGRAMMING

POTHURAJU SAI CHARAN TEJ
192465048RC

Language: Python C C++ Java

QUESTIONS

Q1
Coin Change vs Knapsack Problem (DP Comparison)
100 marks

0/1 answered

Q1 Coin Change vs Knapsack Problem (DP Comparison)100 marks

Given coins and items, solve Coin Change and Knapsack problems using DP. Compare recurrence relations and table structures. Analyze differences in constraints. Evaluate complexity. Discuss scalability. Generate insights on DP modeling.

Your Code

```
1 n = int(input())
2 coins = list(map(int, input().split()))
3 amount = int(input())
4 dp = [float('inf')] * (amount + 1)
5 dp[0] = 0
6 for i in range(1, amount + 1):
7     for coin in coins:
8         if coin <= i:
9             dp[i] = min(dp[i], dp[i - coin] + 1)
10 if dp[amount] == float('inf'):
11     coin_result = -1
12 else:
```

Input

```
3
1 2 5
11
3
```

Output

```
Minimum Coins = 3
Maximum Knapsack Value = 9
```